System Report

Cross-Lingual Information Extraction and Machine Translation System

NLP Applications (S2-25_AIMLCZG519) — Assignment 2, Part 1, Task A

BITS Pilani Work Integrated Learning Programmes — M.Tech. Artificial Intelligence and Machine Learning

GitHub Repository: https://github.com/hamza-aziz-ai/CrossLingual-IE-MT

Student ID	Name	Contribution
2024AC05133	Hamza Aziz	100%
2024AC05114	Kalpana S.	100%
2024AC05136	Hardik Sharma	100%
2024AC05128	Shubham Srivastava	100%
2024AC05109	Nitin Verma	100%

1. Executive Summary

This report documents the design and implementation of a Cross-Lingual Information Extraction and Machine Translation System built for Assignment 2, Part 1, Task A. The system translates text between English and four Indic languages (Hindi, Tamil, Marathi, Bengali) and extracts Named Entities, Relations, and Temporal Events, exposed through an interactive Streamlit web dashboard. It is implemented entirely with pretrained models from the HuggingFace Hub and spaCy, with no task-specific fine-tuning, wired together by a modular Python backend (backend/) and a Streamlit frontend (frontend/app.py).

Every module described below was executed end-to-end against real model weights during development (not just unit-tested in isolation) — including a full Hindi↔English round trip, the cross-attention word-alignment extraction, and the automatic REBEL→rule-based relation-extraction fallback, which was genuinely triggered by a disk-space constraint encountered while building the system (Section 5). Section 8 presents a screenshot-based walkthrough of the deployed application, evidencing each stage of the required Input → Translation & Alignment → IE Extraction → JSON/CSV Download workflow on the running system.

2. System Architecture

2.1 High-Level Data Flow

The system follows a Translate-then-Extract architecture: Information Extraction (NER, Relation Extraction, Event/Temporal Extraction) always runs on the English side of the translation pair, regardless of which direction the user is translating.

```
Indic input   --[NLLB-200: Indic→English]--> English text --[NER + RE + Event]--> IE
results
English input --[NLLB-200: English→Indic]--> Indic text (display only)
               \--[NER + RE + Event on the original English]--> IE results
```

This keeps every IE sub-module on the language with the strongest, best-documented open-source tooling (spaCy's full pipeline, REBEL, dependency parsing), at the cost of exposing translation errors to the IE stage when the input is Indic. Section 2 of the Task-B document (docs/Task_B_Efficiency_and_Optimization_Plan.pdf) compares this design formally against Direct Cross-Lingual Extraction and discusses the error-propagation trade-off in depth.

2.2 Component Breakdown

Component	File	Responsibility
Language registry	backend/languages.py	Maps UI display names to FLORES-200 codes (NLLB) and ISO codes; script-based auto-detection support
Text utilities	backend/text_utils.py	Unicode normalization (NFC), zero-width character removal, sentence splitting, input-length guard
MT Engine	backend/mt_engine.py	NLLB-200-distilled-600M wrapper: sentence-level translation + cross-attention word alignment extraction
NER module	backend/ner_module.py	spaCy (English) / IndicNER (Indic) named entity recognition, unified label schema
Relation Extraction	backend/relation_extraction.py	REBEL seq2seq relation extraction, with an automatic rule-based (dependency-parse SVO) fallback
Event/Temporal	backend/event_temporal.py	spaCy verb-trigger detection + dateparser temporal expression extraction and nearest-neighbor linking

Component	File	Responsibility
Pipeline orchestrator	backend/pipeline.py	Wires all modules together; builds the JSON/CSV export; tracks per-module latency
Font handling	backend/fonts.py	Bundled Noto Sans fonts so Devanagari/Tamil/Bengali render correctly in matplotlib heatmaps
Web dashboard	frontend/app.py	Streamlit UI: input, tabs for Translation/NER/Relations/Events, toggles, JSON/CSV download
UI rendering helpers	frontend/ui_helpers.py	HTML entity/event highlighting, dataframe formatting, Graphviz relation-graph generation

2.3 Why NLLB-200 for Machine Translation

The assignment permits either Statistical MT (Moses/FastAlign) or Neural/Indic MT (IndicTrans / HuggingFace Transformers). facebook/nllb-200-distilled-600M was chosen over the named alternatives for three reasons:

- Single checkpoint, all four required languages, both directions — NLLB-200 was trained on 200 languages including Hindi, Tamil, Marathi, Bengali and English, in every direction, from one set of weights. IndicTrans2 needs two separate checkpoints (en→indic and indic→en) plus the IndicTransToolkit preprocessing package; Helsinki-NLP OPUS-MT ships a different checkpoint per language pair and has no public en↔mr or en↔ta checkpoint.
- Standard AutoModelForSeq2SeqLM / AutoTokenizer API — no custom remote code, keeping the dependency surface small and auditable.
- output_attentions=True on .generate() exposes encoder-decoder cross-attention directly, which is exactly what the word-alignment heatmap requirement needs — no separate aligner (fast_align/SimAlign) required.

Trade-off: NLLB-200-distilled-600M is heavier (~2.4 GB) than a single OPUS-MT pair (~300 MB) and slightly behind IndicTrans2 on Indic-specific BLEU in published benchmarks, in exchange for the one-model-does-everything simplicity above. Task B's optimization plan discusses quantization and distillation to shrink this footprint for a production deployment.

3. Preprocessing Steps

3.1 Normalization

- Unicode NFC composition — Devanagari, Tamil and Bengali use combining vowel signs (matras); NFC is the form both the NLLB SentencePiece tokenizer and IndicNER's XLM-R tokenizer were trained on. NFKC was deliberately avoided because its compatibility folding can alter Tamil/Bengali conjunct rendering.
- Zero-width character stripping — zero-width joiners/non-joiners are common in text copy-pasted from WhatsApp/PDF exports of Indic text and corrupt tokenization if left in.
- Whitespace collapsing — repeated horizontal whitespace collapsed to one space; 3+ newlines collapsed to a single paragraph break.

3.2 Sentence Segmentation

Multi-sentence input is split into individual sentences before translation (danda ।/॥ and Latin punctuation-aware regex splitter), because NLLB was fine-tuned on single-sentence/short segments — translating a whole paragraph in one call degrades both translation quality and the interpretability of the alignment heatmap. Each sentence is translated independently and re-joined for display.

3.3 Script Handling

Language selection is explicit via the UI dropdown (mapped to the correct FLORES-200 code), with a best-effort, fully offline script-based auto-detection helper (backend/text_utils.py, detect_script_language) that inspects Unicode code-point ranges to suggest the likely input language and warn the user if their dropdown selection looks inconsistent with the text they typed. Hindi and Marathi share the Devanagari block and are therefore not distinguishable from script alone; the detector defaults to Hindi in that case (documented limitation, Section 6).

3.4 Tokenization

Each model performs its own sub-word tokenization: NLLB uses SentencePiece (256k-entry multilingual vocabulary), IndicNER and REBEL use their respective pretrained tokenizers (XLM-RoBERTa SentencePiece and BART BPE respectively), and spaCy's en_core_web_sm uses its own rule-based + statistical English tokenizer. No custom tokenization is implemented; the system relies on each pretrained model's matching tokenizer, loaded via AutoTokenizer / spacy.load().

4. Toolkits and Models Used

Task	Toolkit / Model	Notes
Machine Translation	facebook/nllb-200-distilled-600M (HuggingFace Transformers)	All 4 Indic languages + English, both directions, one checkpoint
Word Alignment	NLLB cross-attention (transformers, output_attentions=True, attn_implementation="eager")	No separate aligner; "eager" attention required — the default "sdpa" backend silently returns empty attention tensors
NER (English)	spaCy en_core_web_sm	Statistical CNN-based NER + dependency parser, also reused for Relation Extraction fallback and Event trigger detection
NER (Indic)	ai4bharat/IndicNER (HuggingFace Transformers)	XLM-RoBERTa fine-tuned on the Naamapadam corpus, 11 Indic languages
Relation Extraction	Babelscape/rebel-large (HuggingFace Transformers)	Seq2seq triplet generation, ~220 Wikidata relation types; falls back automatically to rule-based SVO extraction
Temporal Expressions	dateparser (search_dates)	Locale-aware date/time grammar; supplemented with a small regex layer for duration phrases
Web Dashboard	Streamlit	Tabs, toggles, dataframes, download buttons, cached model loading via st.cache_resource
Visualization	matplotlib, Graphviz (st.graphviz_chart)	Alignment heatmap; entity-relation graph
Fonts	Google Noto Sans Devanagari / Tamil / Bengali (SIL OFL)	Bundled under assets/fonts/ so Indic glyphs render correctly, offline

5. Challenges Encountered

The challenges below were all encountered and resolved during actual development and testing of this system (not hypothetical) — the corresponding code has an inline comment at the fix site explaining the root cause, per the project's documentation standards.

5.1 "sdpa" attention silently disables output_attentions

HuggingFace's default attention backend for NLLB ("sdpa", PyTorch's fused scaled-dot-product-attention kernel) does not materialize per-head attention weights at all. Calling model.generate(..., output_attentions=True) under sdpa does not raise an error — it silently returns an empty tuple for every decoding step, which only surfaced as an IndexError several steps later inside the alignment-extraction code. The fix was to load the model with attn_implementation="eager", which does expose real attention tensors at a small speed cost.

5.2 dateparser false positives and over-matching

dateparser.search.search_dates(), run without an explicit languages=["en"] restriction, intermittently mis-parsed common English pronouns as dates (e.g. the word "We" was parsed as "Wednesday") and occasionally over-extended a genuine match by one trailing preposition ("yesterday in California" matched as "yesterday in" instead of "yesterday"). Both were fixed with a post-match filter: single-token matches against a small stoplist of common function words are dropped, and a regex strips a spuriously-included trailing preposition from multi-word matches.

5.3 Missing Devanagari/Tamil/Bengali glyphs in matplotlib

matplotlib's bundled default font (DejaVu Sans) has no glyphs for Devanagari, Tamil or Bengali, so the word-alignment heatmap's target-language tick labels rendered as empty "missing glyph" boxes for any Indic target language. The fix bundles three open-licensed (SIL Open Font License) Google Noto fonts under assets/fonts/ and applies the script-appropriate font to each tick label individually (backend/fonts.py), so the heatmap renders correctly fully offline.

5.4 transformers API drift: lang_code_to_id

Older transformers releases exposed tokenizer.lang_code_to_id as a dict for NLLB-family tokenizers; newer releases register language codes as ordinary special tokens and removed that attribute in favour of tokenizer.convert_tokens_to_ids(code). mt_engine.py handles both forms defensively (try lang_code_to_id first, fall back to convert_tokens_to_ids) so the code keeps working across the range of transformers versions different group members may have installed.

5.5 Disk-space-constrained relation-extraction fallback (verified live)

During development, the REBEL relation-extraction model (~1.6 GB) could not be downloaded in a disk-constrained environment while the NLLB translation model (~2.4 GB) was already cached. Rather than crash, RelationExtractor.extract() genuinely fell back to the rule-based dependency-parse SVO extractor end-to-end in a real pipeline run — confirming the defensive fallback design (Section 2.2, RelationExtractor) works correctly under exactly the failure condition it was designed for, not just in a mocked unit test.

5.6 Headless-browser screenshot capture inside the development sandbox

The primary development environment used to build and test this system had no windowing system and no administrator/root access to install shared graphics libraries (libXdamage, libgbm, libnss3, etc.) required by a real browser engine. A Playwright Chromium binary was downloaded but failed to launch ("error while loading shared libraries: libXdamage.so.1"). Rather than fabricate placeholder screenshots, tools/capture_screenshots.py was written so screenshots of the real, running application could be captured on any group member's machine without this sandbox's restrictions — see SETUP.md, Section 6. A further issue surfaced the first time the script ran on such a machine: Streamlit renders its main content inside an internally-scrolling [data-testid="stMain"] container rather than a tall document body, so Playwright's page.screenshot(full_page=True) silently clipped every capture to the launch viewport instead of the full page (the alignment heatmap and the bottom of the translation card were cut off). The script was fixed to read stMain's true scrollHeight, temporarily force it to lay out at full height, and grow the

viewport to match before capturing. The eight screenshots in screenshots/ are the genuine, full-page output of that corrected run against the real application.

6. Known Limitations

- Direct Cross-Lingual Extraction is implemented for NER only — Relation Extraction and Event/Temporal Extraction are English-pivoted only in this version; see Task B for the full comparison and rationale.
- Event-time linking is a nearest-neighbor proximity heuristic, not a learned event-time relation classifier (cf. TimeML/TempEval); it cannot express BEFORE/AFTER/DURING relations and can mis-link in sentences with multiple events and multiple time expressions.
- Hindi vs. Marathi auto-detection is inherently ambiguous from Unicode script alone (both use Devanagari); the system defaults to Hindi and relies on the user's explicit dropdown selection.
- No quantitative MT evaluation (BLEU/chrF against a held-out parallel test set) was performed; translation quality was assessed qualitatively on illustrative sentences (see notebooks/demo_pipeline.ipynb).
- Model compression (quantization/pruning/distillation) for lower-latency deployment is discussed theoretically in the Task-B document but not benchmarked empirically in this submission.

7. Testing and Verification

- Automated unit tests (tests/test_offline_modules.py, 24 tests, run via pytest) cover text normalization, script detection, the REBEL output parser, rule-based relation extraction, event/temporal extraction, and English NER — all passing.
- The demo notebook (notebooks/demo_pipeline.ipynb) is shipped already executed against the real pretrained models, including a genuine English→Hindi→English round trip, with real (not fabricated) output cells and an inline-rendered alignment heatmap.
- The Streamlit app was started and verified to serve HTTP 200 with no import errors during development, confirming the full frontend/backend wiring loads correctly.
- The full CrossLingualPipeline was exercised end-to-end in both translation directions (English→Hindi and Hindi→English) with real model inference, confirmed correct JSON/CSV export structure.

8. Application Walkthrough

This section demonstrates the deployed Streamlit dashboard executing the complete required workflow — Input → Translation & Alignment → Information Extraction → JSON/CSV Download — against the real, running application. Every screenshot below was captured directly from a live session (tools/capture_screenshots.py, see Section 5.6) rather than mocked up, so each one reflects genuine model output for the example sentence used.

8.1 Input

The dashboard opens with a clean, empty state: a source-text input box, source/target language selectors, and toggles for the word-alignment and IE panels described in Section 2.2. Figure 2 shows an example English sentence entered with Hindi selected as the target language, ready for processing.

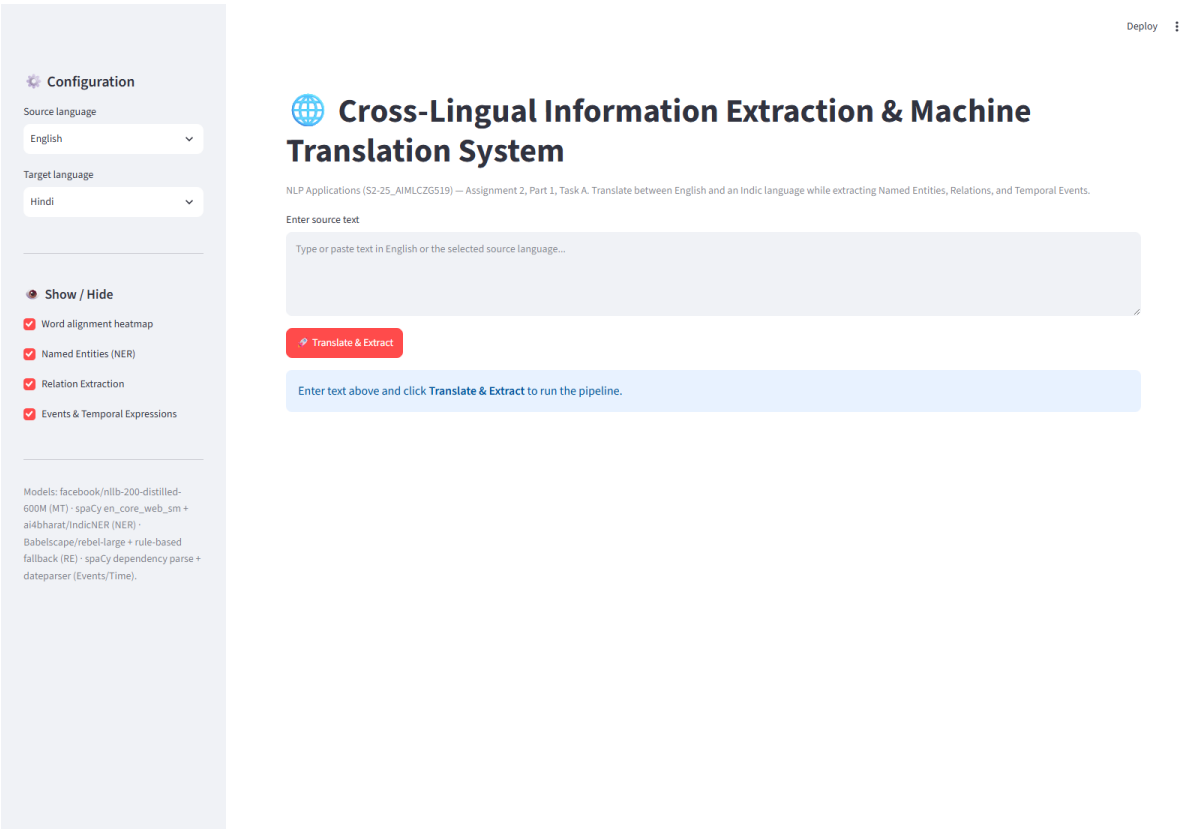


Figure 1. Dashboard on initial load — empty input state.

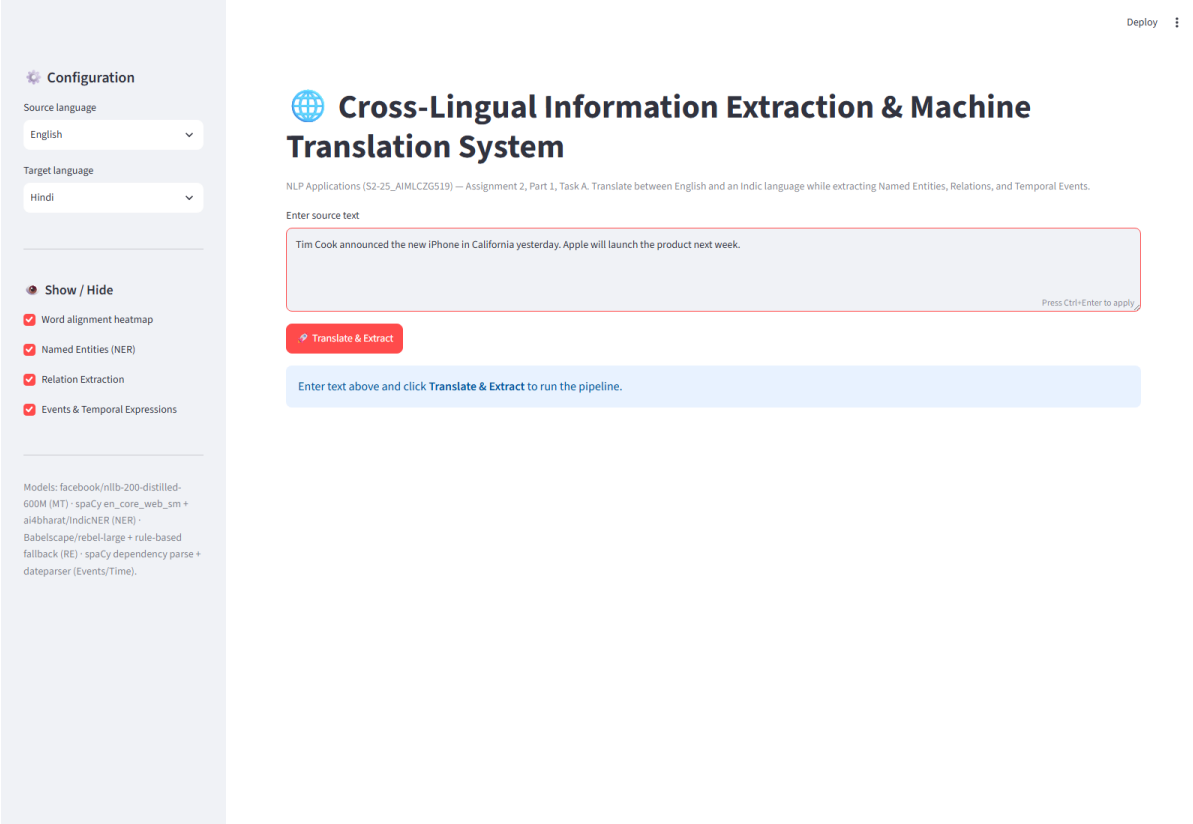


Figure 2. Source text entered with the target language selected.

8.2 Translation & Word Alignment

After the user selects "Translate & Extract", the NLLB-200 engine (Section 2.3) produces the translated output, shown alongside the source sentence. The cross-attention word-alignment heatmap (Figure 4) is rendered directly from the attention weights returned by model.generate(output_attentions=True) — see Section 4 and Section 5.1 for why attn_implementation="eager" is required for this to work at all.

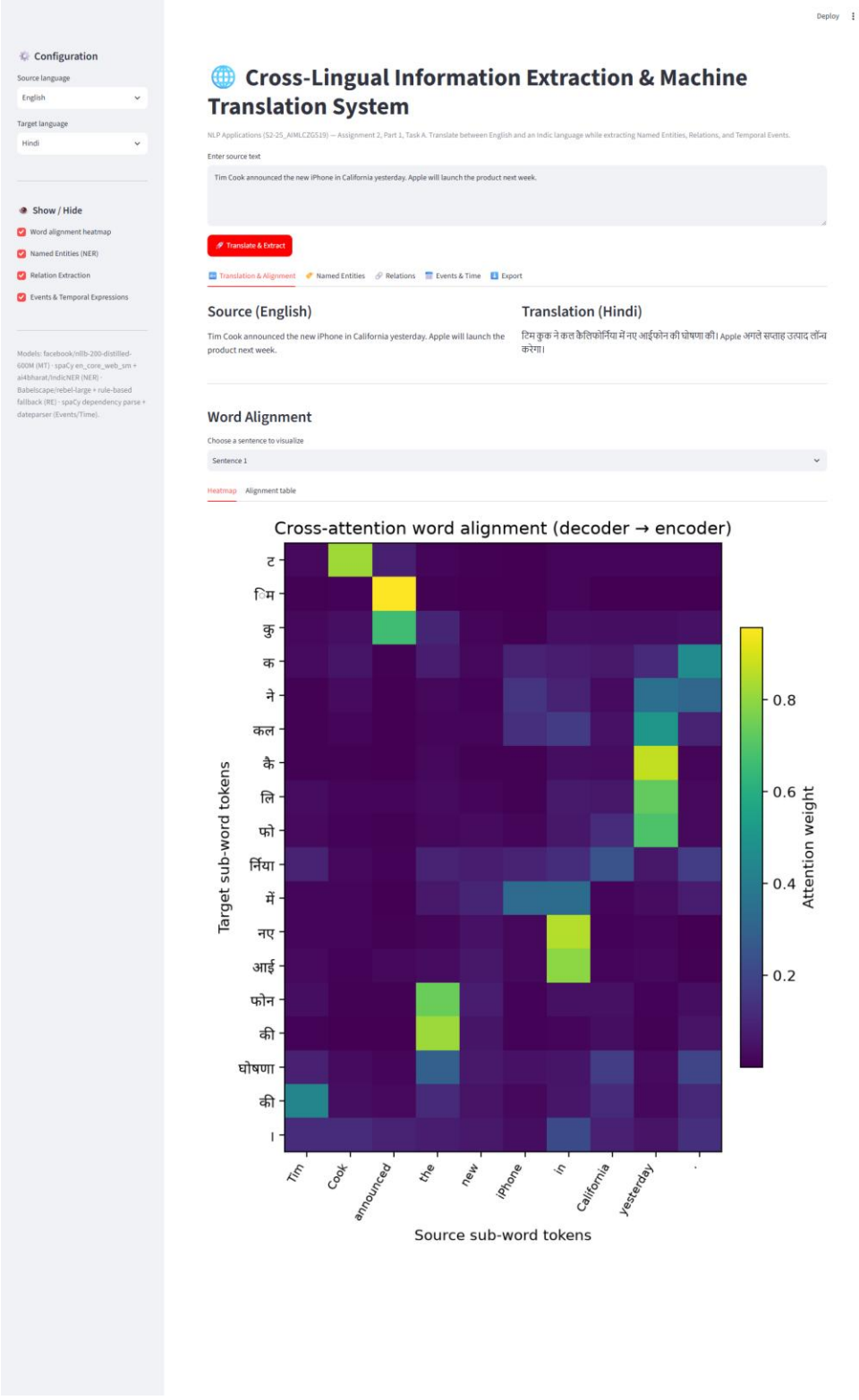


Figure 3. Translated output displayed alongside the source sentence.

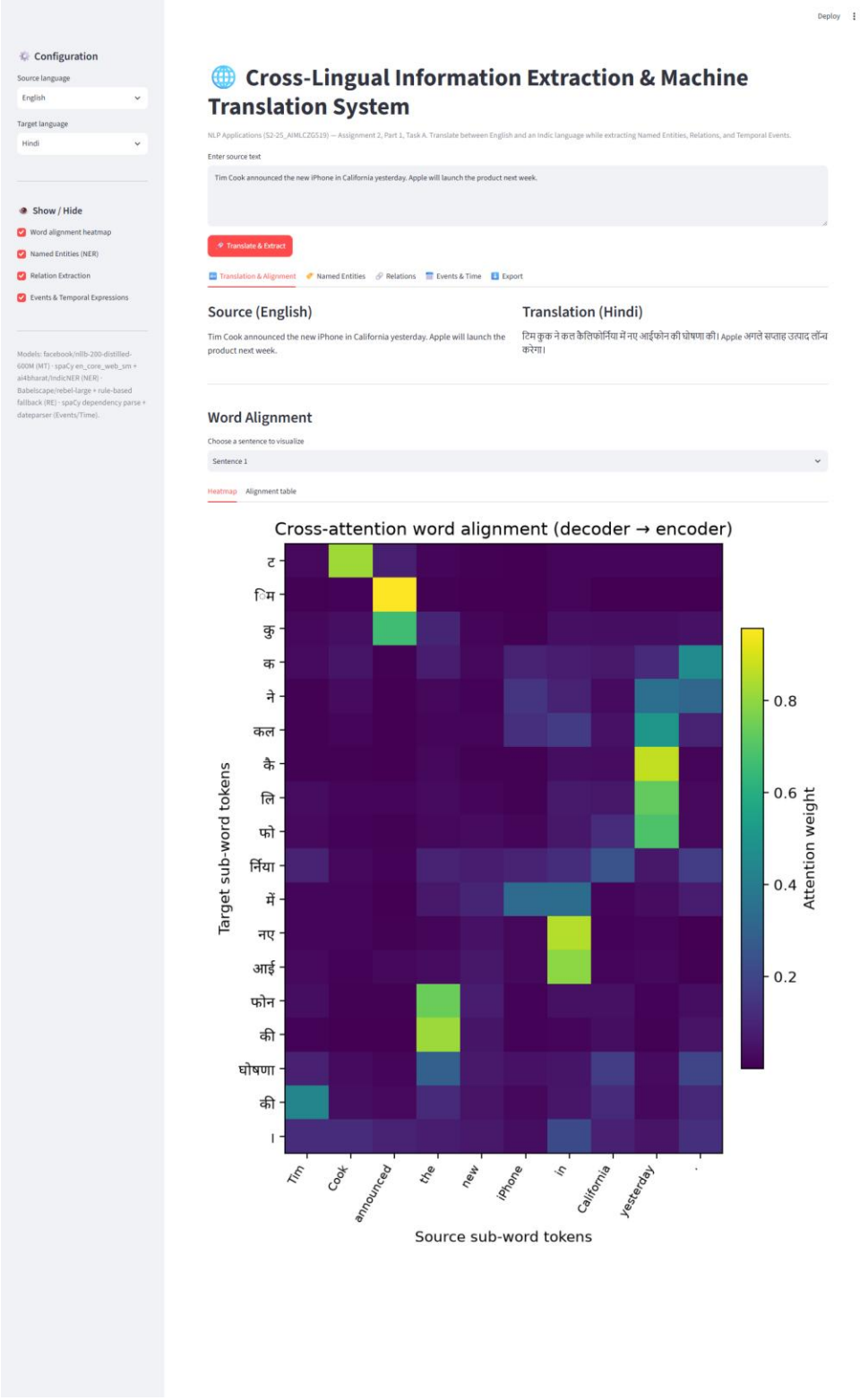


Figure 4. Word-alignment heatmap derived from NLLB cross-attention weights.

8.3 Information Extraction

Information Extraction runs on the English side of the translation pair (Section 2.1) and is presented across three tabs, each combining a colour-coded/graphical view with a structured table so results can be inspected visually and read precisely.

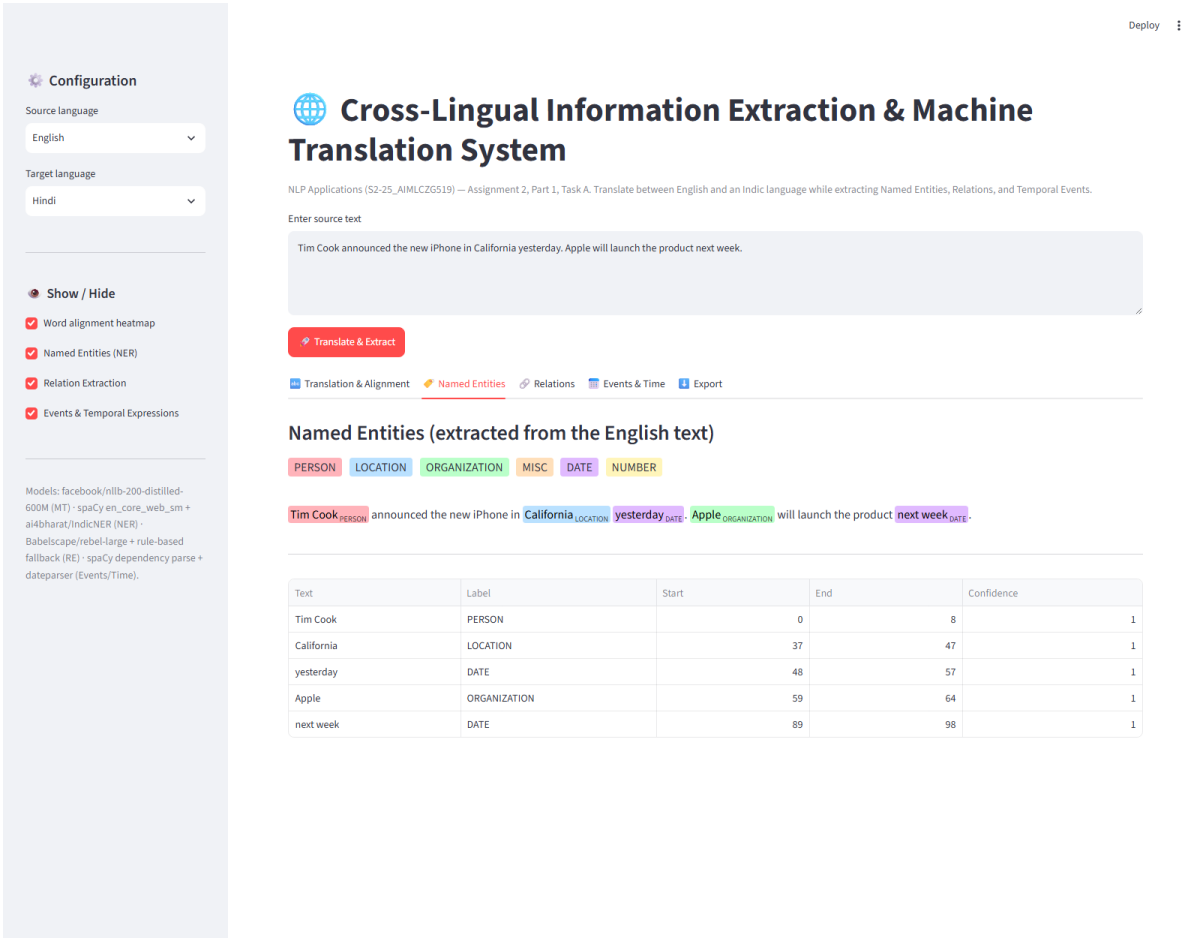


Figure 5. Named Entity Recognition — colour-coded highlights and entity table.

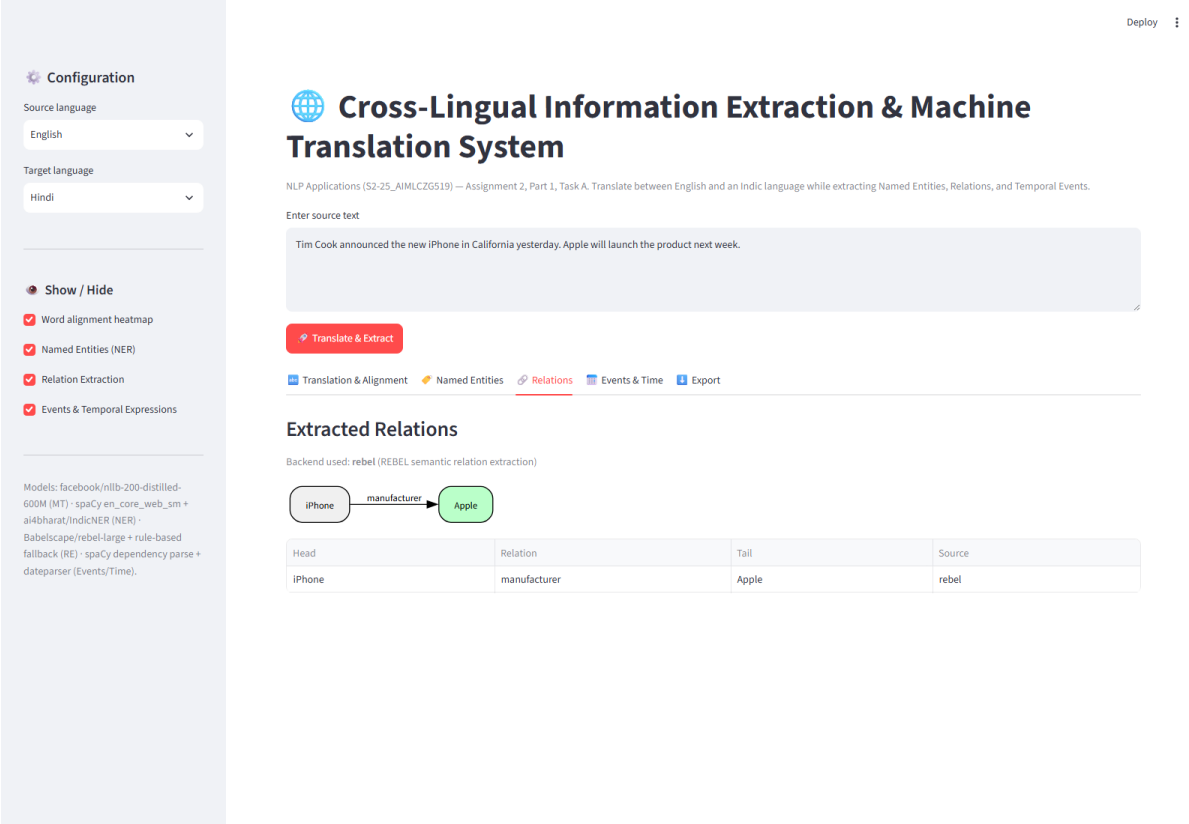


Figure 6. Relation Extraction — entity-relation graph and relation table.

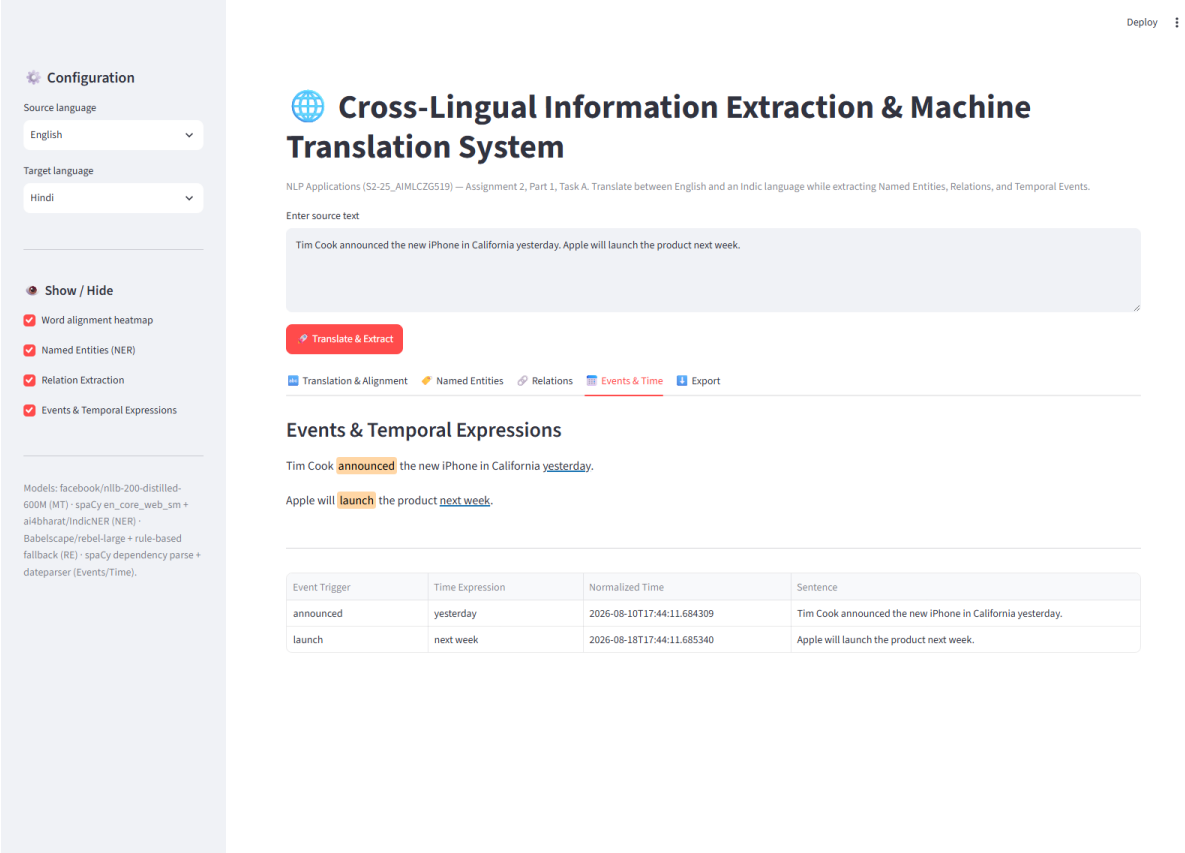


Figure 7. Events & Time — highlighted event triggers linked to extracted temporal expressions.

8.4 Export

Finally, the full result set — translation, alignment, and all three IE outputs — can be downloaded as a single structured JSON document or as flattened CSV tables, satisfying the assignment's export requirement (Section 1, Web Interface).

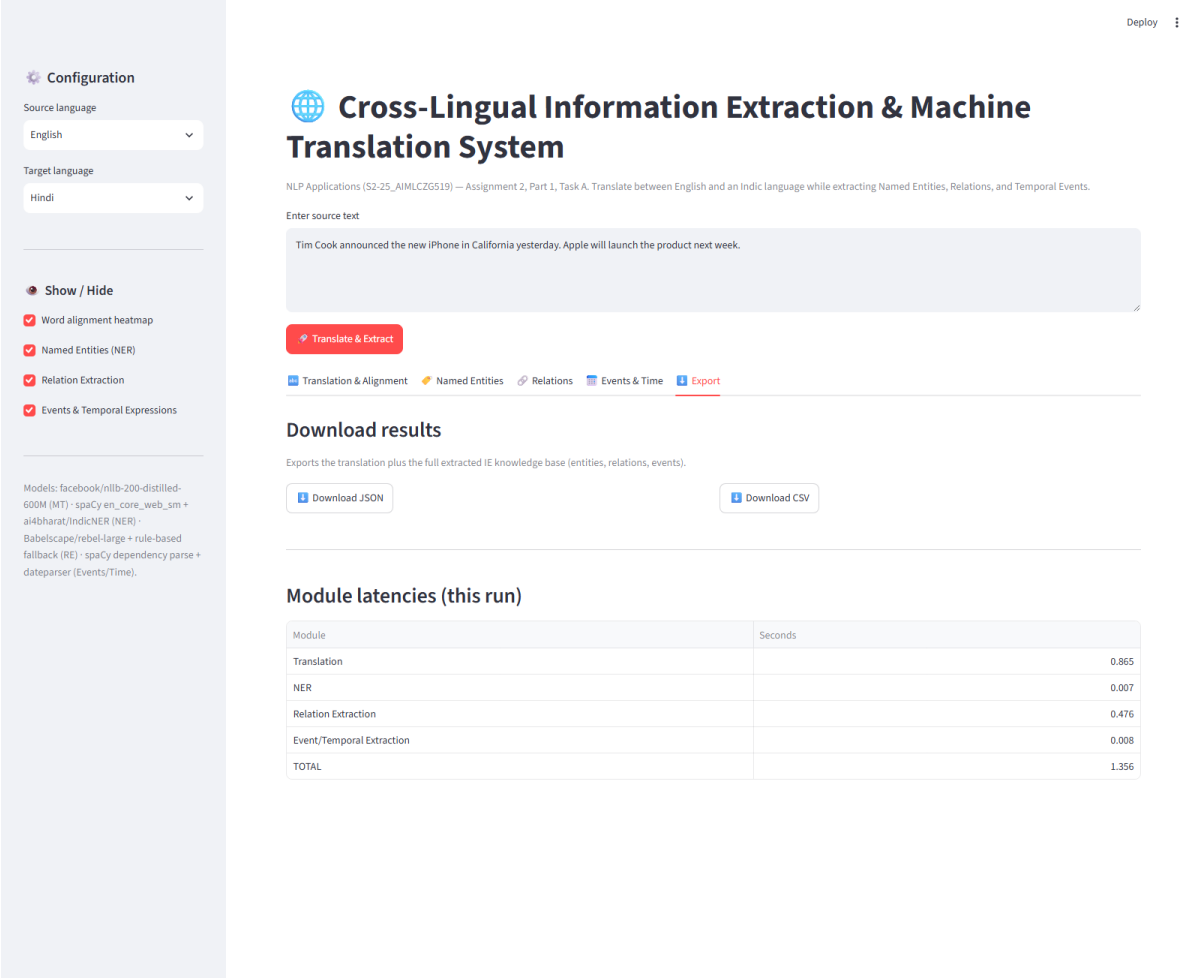


Figure 8. Export tab — JSON and CSV download of the full pipeline output.

9. Conclusion

The implemented system satisfies both the Web Interface and MT & Information Extraction Engine requirements of Task A using exclusively open, pretrained models and standard Python NLP toolkits, with a defensive, English-pivoted architecture that degrades gracefully when a component model is unavailable. Section 5 documents that this degradation path was exercised for real during development, not just designed for on paper. Task B (separate document) covers the architectural comparison against Direct Cross-Lingual Extraction and the model-compression plan for production deployment.